МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНОМУ УНІВЕРСИТЕТІ

“ЛЬВІВСЬКА ПОЛІТЕХНІКА”

Кафедра систем штучного інтелекту



Документація до РР

з дисципліни

«Математичні методи дослідження операцій»

Виконав:
студент групи КН-215
Лозовий О.А.

Викладач:
Шиманський В.М.

Львів – 2023 р.

Тема набору даних: MATICBUSD trades

Приклад з набору даних:

ts	open	high	low	close	volume	rec_count	avg_price	EMA	ADOSC	TRANGE
2022-11-11 15:41:00	1.0209	1.0224	1.0165	1.0165	64454.0	51	1.0197215363	1.0215837394	279505.9690615129	0.0243
2022-11-11 15:42:00	1.0164	1.0203	1.0164	1.0169	173310.0	48	1.0186668807	1.0214939899	178052.0163896361	0.0099
2022-11-11 15:43:00	1.0168	1.0174	1.0142	1.0174	99832.0	47	1.0150476531	1.0212956411	160302.6251702651	0.0114
2022-11-11 15:44:00	1.0178	1.0206	1.0163	1.0163	132244.0	49	1.0192245055	1.0212319139	90160.5901104983	0.0114
2022-11-11 15:45:00	1.0163	1.0163	1.0126	1.0143	58729.0	57	1.0143528359	1.0210202499	14014.8787648155	0.0078
2022-11-11 15:46:00	1.0138	1.0164	1.0133	1.0150	83230.0	50	1.0149083756	1.0208321923	-11638.2034681538	0.0097
2022-11-11 15:47:00	1.0149	1.0222	1.0149	1.0216	148102.0	59	1.0195065428	1.0207914030	-17305.3822481731	0.0075

Кількість записів: >58000

Атрибути (вхідні значення):

- EMA – фінансовий індикатор
- ADOSC – фінансовий індикатор
- TRANGE – фінансовий індикатор

Ціль (вихідне значення):

avg_price – **середня ціна MATIC за останній період часу(1 хв)**

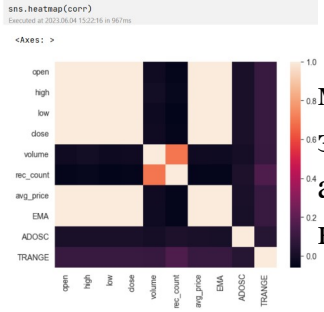
При написанні даної лабораторної роботи я використовува такі види регресій як:

- Linear (ax + b)
- Multiple linear(ax1 + bx2 + cx3 + d)
- Polynomial (ax^3 + bx^2 + cx + d)

Перш ніж починати роботу з регресіями необхідно провести аналіз залежностей(кореляційний аналіз)

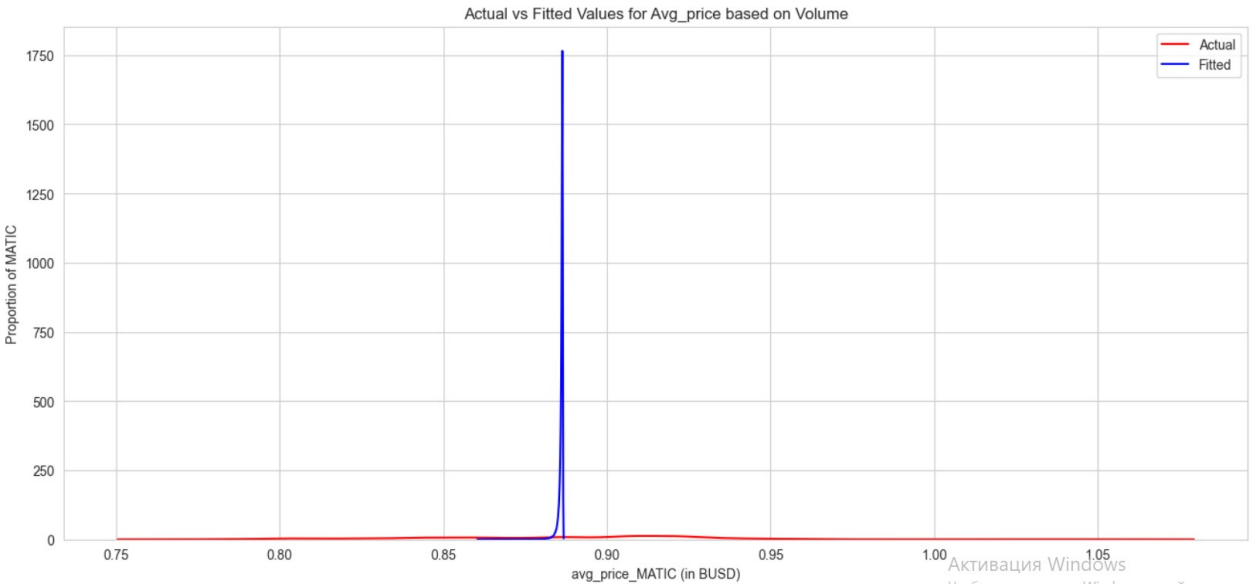
10 rows 10 rows × 10 columns pd.Dataframe CSV

	open	high	low	close	volume	avg_price	EMA	ADOSC	TRANGE
low	0.9998323585	0.9996841822	1.0000000000	0.9998518008	-0.0276155129	0.9999098322	0.9960090258	0.0114686205	0.0968131154
close	0.9997278903	0.9998374931	0.9998518008	1.0000000000	-0.0184016184	0.9999297158	0.9959760796	0.0114328818	0.0983855045
open	1.0000000000	0.9998280144	0.9998323585	0.9997278903	-0.0183061105	0.9998698943	0.9962101768	0.0116055308	0.0983892004
avg_price	0.9998698943	0.9999112962	0.9999098322	0.9999297158	-0.0180726349	1.0000000000	0.9960713869	0.0115056266	0.0984529037
EMA	0.9962101768	0.9961069852	0.9960090258	0.9959760796	-0.0203439193	0.9960713869	1.0000000000	0.0081200224	0.0997802811
high	0.9998280144	1.0000000000	0.9996841822	0.9998374931	-0.0086815000	0.9999112962	0.9961069852	0.0115353316	0.1000790604
ADOSC	0.0116055308	0.0115353316	0.0114686205	0.0114328818	-0.0026214673	0.0115056266	0.0081200224	1.0000000000	0.0486986538
TRANGE	0.0983892004	0.1000790604	0.0968131154	0.0983855045	0.0930392646	0.0984529037	0.0997802811	0.0486986538	1.0000000000
volume	-0.0183061105	-0.0086815000	-0.0276155129	-0.0184016184	1.0000000000	-0.0180726349	-0.0203439193	-0.0026214673	0.0930392646
rec_count	-0.0517300042	-0.0419075302	-0.0610021280	-0.0519125222	0.6937612664	-0.0514037446	-0.0509202431	0.0189904666	0.1560124273



Чудово видно, що стовпці Low, Close, High і Open, не можуть використовуватись, через те, що між ними є залежність. Також глянувши на залежності між Rec_count та avg_price та volume та avg_price, можемо сказати, що це не найкращі стовпці для передбачення.

Продемонструємо це , побудувавши лінійну регресію з використанням Volume:



Вивівши результат у формі графіку, бачимо, що він абсолютно незадовільний.

```
lm.fit(X, Y)
# Find the R^2
print('The R-square is: ', "{:.3f}".format(lm.score(X, Y)))
```

Executed at 2023.06.04 15:22:18 in 18ms

```
The R-square is:  0.000
```

```
mse = mean_squared_error(df['avg_price'], Yhat)
print('The mean square error of price and predicted value is: ', mse)
```

Executed at 2023.06.04 15:22:18 in 18ms

```
The mean square error of price and predicted value is:  0.0019702254968002584
```

Порахувавши похибки, бачимо, що вони досить великі, як для наших значень.

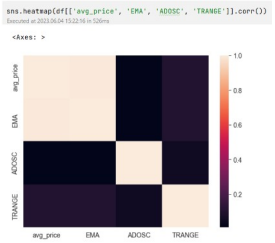
Продовжимо наш кореляційний аналіз:

```
df[['avg_price', 'EMA', 'ADOSC', 'TRANGE']].corr()
```

Executed at 2023.06.04 15:22:16 in 205ms

4 rows × 4 columns pd.Dataframe

	avg_price	EMA	ADOSC	TRANGE
avg_price	1.0000000000	0.9960713869	0.0115056266	0.0984529037
EMA	0.9960713869	1.0000000000	0.0081200224	0.0997802811
ADOSC	0.0115056266	0.0081200224	1.0000000000	0.0486986538
TRANGE	0.0984529037	0.0997802811	0.0486986538	1.0000000000



Розглянувши дані результати, можемо сказати, що EMA є найкращим для передбачення avg_price, проте для Multiple linear regression ми можемо використати всі 3 індикатори.

Розглянемо цю Multiple linear regression на основі EMA, ADOSC і TRANGE.

```
pd.set_option("display.precision", 10)
pd.Series(lm.coef_, index=Z.columns, name='Price').to_frame()
```

Executed at 2023.06.04 15:22:18 in 18ms

3 rows × 1 columns pd.Dataframe

	Price
TRANGE	-0.0120315684
EMA	0.9983980197
ADOSC	0.0000000051

```
print("{:.3f}".format(lm.intercept_))
```

Executed at 2023.06.04 15:22:18 in 69ms

```
0.001
```

```
pd.Series([lm.coef_[0], lm.coef_[1], lm.coef_[2], lm.intercept_], index=['a', 'b', 'c', 'd'], name='Price').to_frame()
```

Executed at 2023.06.04 15:22:18 in 77ms

4 rows × 1 columns pd.Dataframe

	Price
a	-0.0120315684
b	0.9983980197
c	0.0000000051
d	0.0012435388

Порахувавши усі коефіцієнти(a, b, c, d) отримаємо таку формулу:

As we saw above, we should get a final linear function with the structure:

$$Yhat = a + b_1X_1 + b_2X_2 + b_3X_3 + b_4X_4$$

What is the linear function we get in this example?

Avg_price = 0.001 - 0.0120315684 x **TRANGE** + 0.9983980197 x **EMA** + 0.0000000051 x **ADOSC**

Тепер побудуймо графік, та обчислімо похибку:

Let's calculate the R^2:

```
# fit the model
lm.fit(Z, df['avg_price'])
# Find the R^2
print(f'The R-square is: {"{:.3f}".format(lm.score(Z, df.avg_price))}')
```

Executed at 2023.06.04 19:13:35 in 133ms

```
The R-square is: 0.992
```

Let's calculate the MSE.
We produce a prediction:

```
Y_predict_multifit = lm.predict(Z)
```

Executed at 2023.06.04 19:13:35 in 140ms

We compare the predicted results with the actual results:

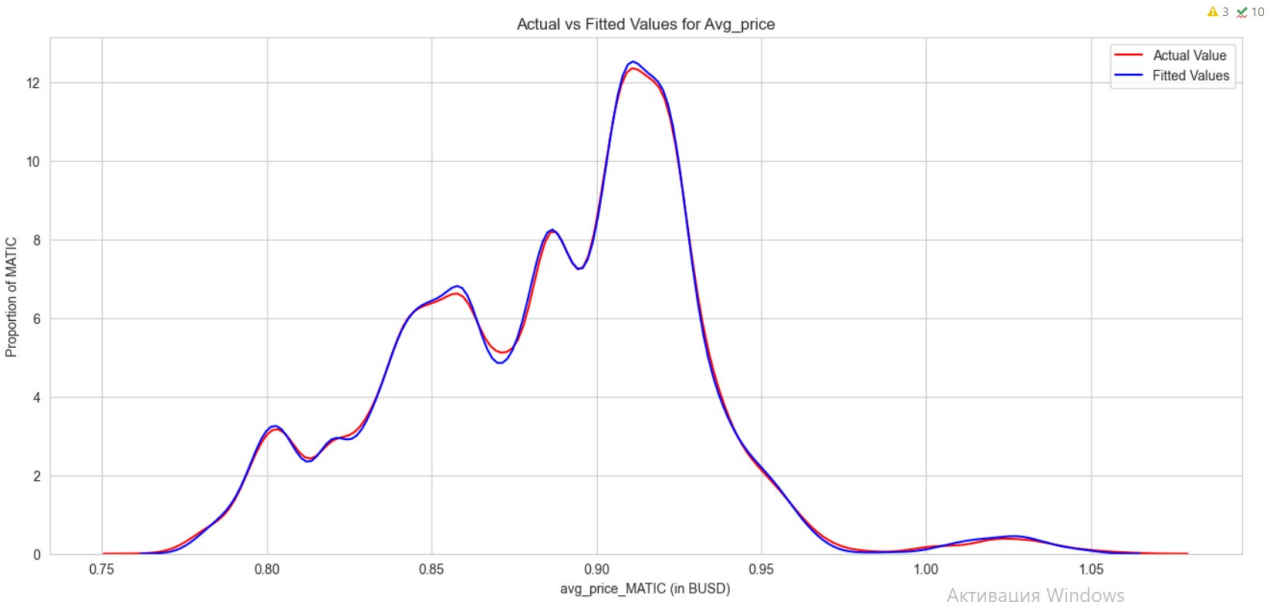
```
print(f'The mean square error of Avg_price and predicted value using multifit is:'
      f' {"{:.6f}".format((mean_squared_error(df.avg_price, Y_predict_multifit)))}')
```

Executed at 2023.06.04 19:13:35 in 124ms

```
The mean square error of Avg_price and predicted value using multifit is: 0.000015
```

Отримана похибка є незначною, а отже ця регресія є більш підходить для нашого випадку

Побудувавши графік, ми можемо побачити, що відхилення незначні.



Тепер побудуймо графік Поліномної регресії

Polynomial Regression and Pipelines

Polynomial regression is a particular case of the general linear regression model or multiple linear regression models.

We get non-linear relationships by squaring or setting higher-order terms of the predictor variables.

There are different orders of polynomial regression:

Quadratic - 2nd Order

$$Yhat = a + b_1X + b_2X^2$$

Cubic - 3rd Order

$$Yhat = a + b_1X + b_2X^2 + b_3X^3$$

Higher-Order:

$$Y = a + b_1X + b_2X^2 + b_3X^3....$$

Для цього необхідно обрахувати її.

```
x = df['EMA']
y = df['avg_price']
```
Executed at 2023.06.04 19:13:35 in 100ms

```
# Here we use a polynomial of the 3rd order (cubic)
f = np.polyfit(x, y, 3)
p = np.poly1d(f)

for i in p:
    print(round(i, 3))
```
Executed at 2023.06.04 19:13:35 in 82ms

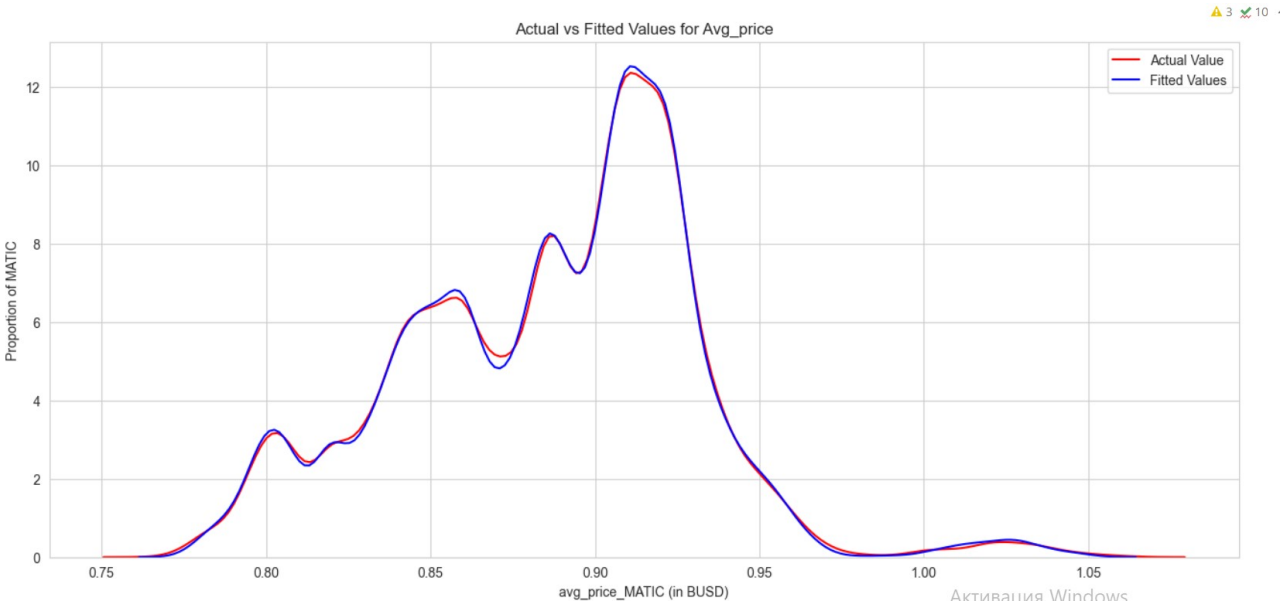
```
-0.034
0.051
0.987
-0.005
```

Add Code Cell | Add Markdown Cell

```
pd.Series([p[3], p[2], p[1], p[0]], index=["Cubic","Quadratic","Linear","0"], name='Coefs').to_frame()
```
Executed at 2023.06.04 19:13:35 in 312ms

4 rows × 1 columns pd.Dataframe

	Coefs
Cubic	-0.0337782211
Quadratic	0.0512962766
Linear	0.9871545664
0	-0.0054528733



З отриманого графіку видно, що поліномна модель теж хороша для передбачення avg_price.

Але яка ж краща? Multiple linear чи Polynomial?

Multiple linear

```
print(f'The mean square error of Avg_price and predicted value using multifit is:'
      f' {"{:.10f}".format((mean_squared_error(df.avg_price, Y_predict_multifit)))}')
```

Executed at 2023.06.04 19:19:28 in 111ms

```
The mean square error of Avg_price and predicted value using multifit is: 0.0000154297
```

Polynomial

```
print(f'The mean square error of Avg_price and predicted value using multifit is:'
      f' {"{:.10f}".format((mean_squared_error(df.avg_price, p(x))))}')
```

Executed at 2023.06.04 19:19:31 in 24ms

```
The mean square error of Avg_price and predicted value using multifit is: 0.0000154405
```

З даних середньоквадратичних похибок видно, що Multiple linear regression все ж трішки краще передбачає значення avg_price ніж Polynomial regression.

Висновок: Для передбачення avg_price найкраще використовувати Multiple linear regression з атрибутами ADOSC, TRAMGE та EMA. Це дає найменшу середню квадратичну похибку, а отже найточніший результат